

PORTFOLIO TRACKER

Comprehensive Codebase Architecture, File Directory Inventory & Usage Analysis

Project Title: Real-Time Multi-Asset Wealth & Portfolio Tracker • **Academic Year:** 2025–2026
Author: Mr. Anshul Ramesh Agrawal (TYBSc Computer Science • Seat No: 01 • Roll No: 01)
Institution: Vidyavardhini's A.V. College, University of Mumbai • **Guide:** Prof. Shrimathi Narayanan
Stack: Python 3.12 (Flask REST API), Supabase PostgreSQL, Vanilla JS/CSS3, Chart.js, Render Cloud

1. Executive Summary & Purpose of This Document

This document provides an exhaustive, component-by-component explanation of all files and directories comprising the **Portfolio Tracker** project. It clarifies **where** each file is used, **why** it was designed, and its exact **operational status** (actively used at runtime, retained as modular architecture source code, used for deployment/infrastructure, or preserved as academic verification and documentation artifacts).

A critical architectural insight of this codebase is the **dual-layer presentation and modular design**: The project provides both clean, decoupled modules (in `routes/` for backend and `js/` for frontend) and consolidated/production-bundled single-file builds (`PORTFOLIO TRACKER.html` with inline scripts and `app.js`). This ensures maximum maintainability for development while guaranteeing resilient zero-dependency browser execution.

2. Master File Taxonomy & Usage Status

The table below categorizes every file in the project, detailing its operational role and whether it is executed actively at runtime or maintained for reference/architecture.

File / Directory	Layer / Category	Where & Why It Is Used	Operational Status
PORTFOLIO TRACKER.html	Frontend UI / SPA	Main single-page web app served by Flask at /. Contains all HTML markup, views, modals, charts container, and standalone inline script.	ACTIVE (Primary Runtime)
style.css	Design System	Complete application styling (~290 KB): dark-theme glassmorphism, responsive CSS grid/flexbox, custom animations, print media rules for PDF reports.	ACTIVE (Primary Runtime)
app.py	Backend Entry Point	Flask application entry point. Loads env, configures secure sessions, registers 5 route blueprints, serves static assets, and hosts /admin.	ACTIVE (Primary Runtime)
app.js	Frontend Reference	Monolithic bundled JavaScript file (~247 KB) containing unified client logic (storage, portfolio CRUD, analytics, charts, alerts, IPOs).	REFERENCE (Consolidated Source)
routes/ (6 files)	Backend API Blueprints	Modular Flask blueprints separating backend concerns: <code>__init__.py</code> , <code>auth.py</code> , <code>holdings.py</code> , <code>prices.py</code> , <code>ipo.py</code> , <code>ai_copilot.py</code> .	ACTIVE (Modular Core)
js/ (7 files)	Frontend Modules	Clean modular frontend source scripts: <code>state.js</code> , <code>portfolio.js</code> , <code>analytics.js</code> , <code>main.js</code> , <code>effects.js</code> , <code>alerts.js</code> , <code>ipo.js</code> .	MODULAR SOURCE (Dev Architecture)
FINAL_PROJECT_REPORT.html	Academic Report	Interactive project report dashboard served at /final-report and /project-report. Houses StarUML diagrams, methodology, and test metrics.	ACTIVE (Report Route)

FINAL_PROJECT_DOCUMENTATION.md	Documentation	Markdown academic project report submitted for University of Mumbai Semester VI (TYBSc CS, Paper USCSP605).	DOCUMENTATION (Academic Spec)
final docs.pdf	Academic Artifact	Compiled master documentation PDF submitted for university examination, external viva, and academic evaluation.	SUBMISSION ARTIFACT (Official PDF)
requirements.txt	Dependencies	Defines Python dependencies (Flask, Gunicorn, requests, python-dotenv, Supabase) for local environments and Render cloud builds.	ACTIVE (Build & Deploy)
render.yaml & Procfile	DevOps / Deployment	Render Cloud Infrastructure-as-Code and Gunicorn WSGI startup configuration. Declares python version, build commands, and port bindings.	ACTIVE (Cloud Production)
env	Configuration / Secrets	Local environment file containing API keys (Supabase, Groq, Google OAuth, SMTP). Loaded by app.py via load_env_file().	ACTIVE (Local Config)
Screenshots/ & demo_results/	Verification Proof	Development test outputs, responsive UI captures, and 30/30 API test suite validation logs for viva and academic audit.	TEST ARTIFACTS (Audit Proof)
Logo & preview assets (.png)	Media & Branding	Visual assets: logo.png, logo_icon.png (favicon/header), and 5 high-res module preview cards used in reports and docs.	ACTIVE (Brand & Media)
welcomed_users.json	Onboarding State	Persistent JSON store of user emails that have received the onboarding welcome email to prevent duplicate dispatches upon reboot.	ACTIVE (Runtime State)

3. Detailed File-by-File Analysis & Architectural Rationale

Below is an in-depth breakdown of each component layer, explaining exactly why each file was created, where it connects within the application lifecycle, and how the modules interact.

3.1 Core Web Presentation & Styling

- **PORTFOLIO TRACKER.html (Main Single Page Application):**

This is the heart of the user experience. It contains the complete semantic HTML5 structure for the application, including the dark-veil canvas background, hero landing section with live metric badges, authentication modals (login, sign-up, Google OAuth button), portfolio net worth dashboard, asset allocation donut charts, real-time holdings data table, trade execution modal (buy, sell, partial sell), IPO tracking tabs, and the AI Copilot chat drawer. **Where used:** Served automatically by app.py at the root route (/). It can also be opened standalone in any web browser.

Why used: Delivering a responsive, zero-broker-credential interface where retail investors can track multi-asset wealth in Indian Rupees (₹).

- **style.css (Design System & Micro-Interactions):**

A 290 KB custom stylesheet implementing institutional-grade dark mode glassmorphism. It establishes the design tokens (HSL CSS variables, typography pairings of *JetBrains Mono* for financial figures and *Plus Jakarta Sans* for interfaces), responsive layouts (mobile, tablet, desktop), TradingView chart containers, badge color-coding (+ve emerald green, -ve rose red), custom scrollbars, and dedicated @media print rules that reformat the live dashboard into a clean, white-background valuation statement for client printing and PDF export.

Where used: Linked directly in PORTFOLIO TRACKER.html and served statically by Flask.

Why used: To provide a visually stunning, premium fintech aesthetic without relying on bulky CSS frameworks.

3.2 Backend Engine & Modular Blueprints (routes/)

The backend is developed with Python 3.12 and Flask. Rather than putting all endpoints in a single monolithic script, the logic is cleanly decoupled into **Flask Blueprints** within the routes/ package:

- **routes/__init__.py (Shared Infrastructure & Database Factory):**

Initializes the Supabase PostgreSQL client (create_client) using environment variables. Establishes in-memory fallback caches (_LOCAL_HOLDINGS_CACHE, _LOCAL_USER_PANS, _LOCAL_IPO_APPS) to ensure 100% application uptime even during database cold-starts or network interruptions. Implements cryptographic security helpers: hash_password() using **PBKDF2-HMAC-SHA256** with 100,000 iterations and random 16-byte cryptographic salt, alongside constant-time verify_password().

Where used: Imported by all other route blueprints to share DB state and avoid circular dependencies.

- **routes/auth.py (Authentication & Authorization Blueprint):**

Handles user authentication and security flows. Endpoints: /api/register (account creation with salt/hash), /api/login (credential validation), /api/logout, /api/user (current session status), and /api/google-auth (Google OAuth 2.0 ID token verification via Google API certificates). Also triggers automated onboarding welcome emails via SMTP upon successful registration.

Where used: Registered in app.py via app.register_blueprint(auth_bp).

- **routes/holdings.py (Portfolio Holdings CRUD Blueprint):**

Manages user asset holdings across NSE, BSE, MCX, Mutual Funds, and US Equities. Provides REST endpoints: GET /api/holdings (retrieves user portfolio with computed gains), POST /api/holdings (adds or modifies an asset holding), and DELETE /api/holdings/<id> (removes an asset). Persists data directly to Supabase PostgreSQL with automated fallback to the local session cache.

Where used: Registered in app.py via app.register_blueprint(holdings_bp).

- **routes/prices.py (Real-Time Price & Market Data Engine):**

Resolves live Current Market Prices (CMP) and historical candle data. Features a multi-worker concurrent fetching engine that queries market quote providers with a 45-second in-memory cache. This caching mechanism drastically minimizes outbound latency (< 2s response) and protects against external API rate-limiting.

Where used: Registered in app.py via app.register_blueprint(prices_bp).

- **routes/ipo.py (IPO Intelligence & Allotment Engine):**

Provides comprehensive Indian IPO tracking for both Mainboard and SME offerings. Delivers live bidding status, issue size, price band, opening/closing dates, and real-time Grey Market Premium (GMP). Includes a PAN-based allotment verification module that queries registrar endpoints or verified mock datasets.

Where used: Registered in app.py via app.register_blueprint(ipo_bp).

- **routes/ai_copilot.py (AI Assistant & Financial News Feed):**

Integrates LLM capabilities via Groq API (or intelligent rule-based financial models) to analyze portfolio health, diversification risk, and asset concentration. Also aggregates curated financial RSS news feeds tailored to Indian retail investors.

Where used: Registered in app.py via app.register_blueprint(ai_bp).

3.3 Frontend Modular Architecture (js/) vs. Monolithic app.js

A key design decision in this project is the coexistence of **clean modular source files** in js/ alongside the **consolidated reference script** app.js and the inline script in PORTFOLIO TRACKER.html:

- **js/state.js:** Global state store managing user session, holdings cache, active tabs, and deterministic PRNG (mulberry32).
 - **js/portfolio.js:** Portfolio management logic: holdings table rendering, buy/sell calculations, P&L; mathematics, CSV export, and TradingView chart launching.
 - **js/analytics.js:** Visual analytics engine configuring Chart.js donut charts (asset allocation) and line charts (net worth trajectory).
 - **js/main.js:** Application bootstrap, DOM event listeners, modal controllers, and search autocomplete.
 - **js/effects.js:** UI micro-interactions, GSAP smooth entrance animations, and Dark Veil canvas particle rendering.
 - **js/alerts.js:** Price target alerts, live notifications, and toast notification manager.
 - **js/ipo.js:** IPO listing rendering, filter tabs (Open, Upcoming, Closed), GMP calculation, and PAN allotment verification modal.
 - **app.js (Consolidated Reference):** Houses the entire unified client logic in a single file (~247 KB). Used as a consolidated development reference and offline backup.

Why both exist: js/ provides maintainable, decoupled code for developers, while PORTFOLIO TRACKER.html embeds the bundled script inline to allow users to double-click and run the app offline without CORS or module loader restrictions.

3.4 Academic Documentation, DevOps & Environment Configuration

- **FINAL_PROJECT_REPORT.html:** An interactive, web-based version of the complete academic project report. It showcases the project overview, literature survey, system analysis, StarUML diagrams (ER, Class, Sequence, Activity, Component, Deployment), test cases (30/30 passed), and references. Served dynamically by app.py at /final-report and /project-report.
- **FINAL_PROJECT_DOCUMENTATION.md:** Markdown academic project specification formatted for University of Mumbai submission. Documents project metadata, objectives, technology stack, hardware/software prerequisites, and live production URLs.
- **final docs.pdf:** The compiled, official project documentation PDF (3.39 MB) submitted for university grading and external viva evaluation.
- **requirements.txt:** Python package manifest specifying pinned dependencies: Flask>=3.0.0, gunicorn>=22.0.0, requests>=2.31.0, python-dotenv>=1.0.0, and supabase>=2.0.0.
- **render.yaml & Procfile:** Production deployment configurations for Render Cloud. render.yaml defines the web service environment, Python 3.12 runtime, and environment variables. Procfile specifies the WSGI process startup command: web: gunicorn app:app -b 0.0.0.0:\$PORT.
- **env:** Local environment configuration holding sensitive API keys and secrets (Supabase DB URL, Anon Key, Google OAuth Client ID/Secret, Groq API Key, Flask Secret Key). Read safely by app.py via load_env_file().
- **welcomed_users.json:** Local runtime state tracking user emails that have already received onboarding welcome emails, preventing duplicate emails when the server restarts.
- **Screenshots/ & demo_results/:** Visual proof-of-work captures and test result logs demonstrating responsive design across devices, API test suite verification, and database sync for academic audit.
- **Logo & Preview Assets (.png):** logo.png & logo_icon.png provide application branding and favicon. Preview graphics (portfolio-tracker-preview.png, holdings_preview.png, analytics_preview.png, news_preview.png, ai_preview.png) are embedded in reports, documentation, and metadata cards.

4. Clarification: Used vs. Reference vs. Redundant Files

To clearly address the question of 'which files are used, why, and if any are not used', here is the exact operational status:

Component Category	Files Included	Exact Usage & Reason for Retention
Actively Used at Runtime	PORTFOLIO TRACKER.html style.css app.py routes/*.py env welcomed_users.json logo_icon.png	These files are directly executed when the user runs python app.py or visits the live cloud application on Render. They handle the live web UI, REST API endpoints, database sync, and user sessions.
Active in Build / Cloud	requirements.txt render.yaml Procfile	Used during environment setup (pip) and cloud deployment on Render to build the container, install packages, and start the Gunicorn WSGI server.
Modular Source Architecture	js/state.js js/portfolio.js js/analytics.js js/main.js js/effects.js js/alerts.js js/ipo.js	These represent the clean, modularized frontend architecture. While the current HTML embeds the bundled script inline for standalone zero-server execution, these files are the active modular source code for future frontend builds and code maintenance.
Reference & Consolidated	app.js	This is the monolithic, consolidated reference file combining all frontend JavaScript. It is preserved for reference, debugging, and single-file scripting.

Academic Documentation & Verification	FINAL_PROJECT_REPORT.html FINAL_PROJECT_DOCUMENTATION.md final docs.pdf Screenshots/ demo_results/ Preview .png files	These are academic deliverables and audit proof. They are not part of the runtime execution loop, but are required for University evaluation, project viva, documentation, and external examination.
---------------------------------------	--	--

5. End-to-End System Request Lifecycle

The following operational flow illustrates how the active files interact during a user session:

- 1. Client Access:** User opens the web app. Flask (app.py) serves PORTFOLIO TRACKER.html and style.css.
- 2. Authentication:** User signs in via email/password or Google OAuth. The request routes to routes/auth.py, which verifies credentials against Supabase PostgreSQL using PBKDF2-SHA256, sets an HTTP-only session cookie, and checks welcomed_users.json.
- 3. Portfolio Loading:** Frontend invokes GET /api/holdings. Handled by routes/holdings.py, which queries Supabase with fallback to in-memory cache.
- 4. Live Price Resolution:** Frontend calls GET /api/prices. routes/prices.py queries live market feeds via multi-threaded workers, caches quotes for 45s, and returns latest CMP.
- 5. Interactive Valuation:** Frontend (analytics.js / inline engine) calculates Day's Gain and Net Worth, updating Chart.js donut/line charts and rendering the holdings table.
- 6. Statement Generation:** User clicks 'Print Valuation Statement'. The custom @media print rules in style.css format the table into an institutional PDF report.

6. Summary & Recommendations

Summary: Every file in this directory serves a clear and defined purpose. No files are accidental clutter. The active application runtime is driven by app.py, PORTFOLIO TRACKER.html, style.css, and routes/. The modular scripts in js/ represent high-quality frontend source code, while app.js serves as the consolidated reference. The remaining files provide deployment automation, environment security, and university-grade academic documentation.

Recommendation: Keep all files as structured. When performing future feature additions, update the modular files in routes/ for backend and js/ for frontend to maintain clean separation of concerns.